TRAVAUX PRATIQUES

ECOTRACK : Base de Données Production PostgreSQL & Supabase

Architecture, Performance & Sécurité
Mastère DevSecOps, Data Science, CyberCloud

École INGETIS Paris

VOTRE MISSION

Votre prototype ECOTRACK fonctionne bien en dev avec Supabase. Mais maintenant, il faut passer à l'échelle : 10,000 poubelles connectées générant 1 million de mesures par jour.

Problèmes actuels :

- Dashboard qui ralentit (3-5 secondes de chargement)
- Pas de versioning du schéma
- Pas d'historisation des changements
- Schéma rigide (difficile d'ajouter des métadonnées)
- Aucune stratégie pour gérer 350M+ de lignes

VOTRE DÉFI

Transformer votre base PostgreSQL/Supabase en système de production robuste, performant et sécurisé. Vous allez MESURER, ANALYSER, OPTIMISER et DOCUMENTER.

Pourquoi PostgreSQL/Supabase ?

Avantage	Bénéfice pour ECOTRACK
JSONB natif	Métadonnées IoT flexibles et performantes
Partitionnement déclaratif	Gérer des centaines de millions de lignes
Row Level Security	Sécurité fine au niveau des données
API auto-générée (Supabase)	Développement frontend accéléré
Real-time subscriptions	Dashboard qui se met à jour en temps réel

Règles du Jeu

- Interdiction de copier-coller du code sans le comprendre
- Cherchez dans la documentation PostgreSQL et Supabase
- Mesurez AVANT et APRÈS chaque optimisation
- Justifiez vos choix techniques
- Entraide autorisée, mais chacun code sa solution
- TOUJOURS sauvegarder avant modification structurelle

Ressources Autorisées

- Documentation PostgreSQL : https://www.postgresql.org/docs/

- Documentation Supabase : https://supabase.com/docs
- Votre cours Bases de Données Avancées
- Forums et Stack Overflow (pour comprendre)
- Vos enseignants pendant les séances

PARTIE 1 : AUDIT & DIAGNOSTIC

Objectif : Analyser votre base PostgreSQL existante et identifier les goulots d'étranglement.

PRINCIPE FONDAMENTAL
Jamais d'optimisation sans mesure préalable. Vous devez établir une BASELINE pour prouver l'efficacité de vos optimisations.

Mission 1.1 : État des Lieux de Votre Base

Contexte : Vous devez connaître la taille, le nombre de lignes, les index existants de toutes vos tables.

MISSION : Cartographier votre base Supabase

Votre audit doit répondre à :

1. Quelles sont vos 5 plus grosses tables ?

Piste de recherche : Cherchez 'PostgreSQL pg_tables' et 'pg_total_relation_size'. Les vues système pg_* contiennent toutes les métadonnées.

2. Quelle est la taille totale de votre base ?

Piste de recherche : Cherchez 'PostgreSQL database size pg_database_size'.

3. Combien d'index par table ? Quels types d'index ?

Piste de recherche : Cherchez 'PostgreSQL pg_indexes'. Regardez aussi pg_stat_user_indexes pour voir si les index sont utilisés.

4. Y a-t-il des tables sans clé primaire ?

⚠️ **DANGER**
Une table sans PK pose des problèmes : pas d'identification unique des lignes, difficultés avec les réplications, performances dégradées pour les UPDATE/DELETE.

Piste de recherche : Cherchez 'PostgreSQL find tables without primary key pg_constraint'.

5. Quels sont les types de colonnes utilisés ?

Avez-vous des colonnes TEXT qui devraient être VARCHAR avec limite ? Des INTEGER qui devraient être BIGINT ? Des TIMESTAMP sans timezone ?

À rendre : Fichier audit_postgresql.md avec tableau : nom_table, nb_lignes, taille_data, taille_index, nb_index, a_pk (oui/non)

Validation : Vérifiez vos chiffres en comparant avec l'interface Supabase (Table Editor → taille affichée).

Mission 1.2 : Identifier les Requêtes Lentes

Contexte : PostgreSQL peut logger les requêtes lentes. Supabase expose aussi des métriques de performance.

MISSION : Activer le monitoring PostgreSQL

Avec Supabase, vous avez deux options :

Option A : Via l'interface Supabase

Piste de recherche : Allez dans Database → Query Performance. Supabase affiche les requêtes les plus lentes automatiquement.

Option B : Via PostgreSQL directement

1. Comment activer pg_stat_statements ?

Piste de recherche : Cherchez 'PostgreSQL pg_stat_statements extension'. C'est une extension qui track toutes les requêtes.

2. Quelle requête SQL permet de voir les requêtes les plus lentes ?

Piste de recherche : Cherchez 'PostgreSQL slow queries pg_stat_statements'. Vous devez SELECT depuis pg_stat_statements et ORDER BY total_exec_time.

MISSION : Analyser vos 3 requêtes les plus lentes

Pour CHAQUE requête lente :

- Copiez la requête SQL exacte

- Lancez EXPLAIN ANALYZE dessus
- Notez le temps d'exécution (Execution Time)
- Identifiez le problème : Seq Scan ? Pas d'index ? Jointure coûteuse ?

Piste de recherche : Cherchez 'PostgreSQL EXPLAIN ANALYZE'. La syntaxe est différente de MySQL : plus verbeux mais plus précis.

SIGNAUX D'ALARME dans EXPLAIN
- Seq Scan sur grosse table (scan séquentiel = mauvais)
- Nombre de rows élevé
- Hash Join ou Nested Loop sur millions de lignes
- Buffers: shared hit faible (données pas en cache)

À rendre : Fichier requetes_lentes.md avec : requête SQL, résultat EXPLAIN ANALYZE, temps exécution, diagnostic du problème

Mission 1.3 : Projection de Croissance

Contexte : Passage de 1,000 à 10,000 poubelles dans 6 mois. Anticipez les besoins.

1. Calculez le volume de données sur 1 an avec 10,000 poubelles

10,000 poubelles × 96 mesures/jour × 365 jours = ? lignes dans measurements

2. Quelle sera la taille de la table en GB ?

Une ligne measurements fait environ 60-100 bytes (selon vos colonnes). Faites le calcul.

3. À partir de combien de lignes partitionner une table PostgreSQL ?

Piste de recherche : Cherchez 'PostgreSQL when to partition table'. Il n'y a pas de règle absolue, mais un ordre de grandeur.

4. Le free tier Supabase (500 MB) suffira-t-il ?

Comparez votre projection avec les limites Supabase. Quel plan faudra-t-il ?

À rendre : Document projection.md avec calculs, stratégie de partitionnement envisagée, estimation coûts Supabase

PARTIE 2 : OPTIMISATION PERFORMANCE

Objectif : Accélérer vos requêtes lentes de 10x à 100x avec indexation et partitionnement PostgreSQL.

Mission 2.1 : Indexation Stratégique

Contexte : PostgreSQL supporte plusieurs types d'index : B-tree, Hash, GIN, GiST, BRIN. Vous devez choisir le bon.

MISSION : Comprendre les types d'index PostgreSQL

Recherchez et expliquez :

1. Quand utiliser un index B-tree (par défaut) ?

2. Quand utiliser un index GIN ?

Piste de recherche : GIN = Generalized Inverted Index. Cherchez 'PostgreSQL GIN index use cases'. Important pour JSONB !

3. Quand utiliser un index BRIN ?

Piste de recherche : BRIN = Block Range Index. Très efficace pour les grosses tables avec données triées (ex: timestamp).

MISSION : Optimiser votre requête #1 identifiée en Partie 1

Questions à vous poser :

1. Quelles colonnes sont dans le WHERE ?

Ce sont vos candidats pour index.

2. Y a-t-il un ORDER BY ou GROUP BY ?

Un index peut aussi accélérer le tri.

3. Index simple ou composite ?

Si WHERE col1 = X AND col2 = Y, un index composite (col1, col2) est souvent meilleur.

4. Quel type d'index : B-tree, GIN, BRIN ?

Justifiez votre choix selon le type de donnée et de requête.

Piste de recherche : Cherchez 'PostgreSQL CREATE INDEX'. Syntaxe : CREATE INDEX nom ON table USING type (colonnes);

Validation : Après création de l'index :
1. EXPLAIN ANALYZE doit montrer 'Index Scan' (pas Seq Scan)
2. Le temps d'exécution doit diminuer de 10x minimum
3. Vérifiez avec pg_stat_user_indexes que l'index est utilisé

MISSION : Optimiser les requêtes #2 et #3

Répétez le processus. Attention à ne pas sur-indexer !

RÈGLE D'OR
Chaque index accélère les SELECT mais ralentit les INSERT/UPDATE/DELETE. Créez uniquement les index dont vous avez PROUVÉ l'utilité avec EXPLAIN.

À rendre : Fichier optimisation_index.md : chaque index créé, type choisi, justification, comparatif AVANT/APRÈS, gains en %

Mission 2.2 : Partitionnement Déclaratif PostgreSQL

Contexte : PostgreSQL 10+ supporte le partitionnement déclaratif. Plus simple que l'ancien système par héritage.

MISSION : Concevoir la stratégie de partitionnement

Décisions à prendre :

1. Type de partitionnement : RANGE, LIST ou HASH ?

Piste de recherche : Cherchez 'PostgreSQL declarative partitioning'. Pour des timestamps, RANGE est généralement optimal.

2. Granularité : mensuelle, trimestrielle ?

Réfléchissez :

- Volume de données par partition (éviter trop petites ou trop grosses)
- Politique de rétention (garder combien de temps ?)
- Requêtes typiques (filtrent-elles par mois ? par trimestre ?)

3. Partitionner par measured_at ou autre ?

Sur quelle colonne vos requêtes filtrent-elles le plus souvent ?

MISSION : Implémenter le partitionnement

Étapes :

1. SAUVEGARDER votre table measurements

Créez une copie : CREATE TABLE measurements_backup AS SELECT * FROM measurements;

2. Créer la table partitionnée

Piste de recherche : Cherchez 'PostgreSQL CREATE TABLE PARTITION BY RANGE'. Syntaxe spécifique PostgreSQL.

3. Créer les partitions

Comment créer chaque partition mensuelle ? Quelle syntaxe ?

Piste de recherche : Cherchez 'PostgreSQL CREATE TABLE PARTITION OF'. Chaque partition est une table fille.

4. Migrer les données

Comment transférer les données de measurements_backup vers la table partitionnée ?

5. Tester le partition pruning

Piste de recherche : EXPLAIN doit montrer 'Partitions removed: X' si le pruning fonctionne.

Validation : Lancez EXPLAIN sur une requête filtrée par date. Vous devez voir 'Partitions removed: XX' et une seule partition scannée.

À rendre : Document partitionnement.md : stratégie choisie, étapes réalisées, preuve du partition pruning, difficultés rencontrées

Mission 2.3 : Optimisation Mémoire PostgreSQL

Contexte : PostgreSQL a de nombreux paramètres de performance. Les plus critiques : shared_buffers, work_mem, effective_cache_size.

MISSION : Analyser la configuration actuelle

1. Comment voir la configuration PostgreSQL actuelle ?

Piste de recherche : Cherchez 'PostgreSQL SHOW command'. SHOW ALL affiche tous les paramètres.

2. Quelle est la valeur de shared_buffers ?

C'est la mémoire partagée pour le cache de données. Par défaut souvent trop faible.

3. Quelle est la valeur de work_mem ?

Mémoire par opération de tri/hash. Trop faible = sorts sur disque.

MISSION : Recommandations de configuration

Avec Supabase, la configuration est gérée. Mais comprendre les principes :

1. Combien de RAM a votre instance Supabase ?

Free tier : 0.5 GB. Pro : 1-8 GB selon le plan.

2. Quelle devrait être la taille de shared_buffers ?

Piste de recherche : Cherchez 'PostgreSQL shared_buffers tuning'. Règle : 25% de la RAM sur serveurs dédiés.

3. Quelle devrait être work_mem ?

Piste de recherche : Cherchez 'PostgreSQL work_mem tuning'. Dépend du nombre de connexions simultanées.

NOTE SUPABASE
Avec Supabase, vous ne pouvez pas modifier shared_buffers (géré par la plateforme). Mais vous pouvez utiliser SET work_mem = 'XXX' au niveau session pour des requêtes spécifiques.

À rendre : Fichier config_postgresql.md : valeurs actuelles, recommandations avec justifications, limites Supabase

PARTIE 3 : ARCHITECTURE MODERNE

Objectif : Exploiter JSONB pour la flexibilité et implémenter Row Level Security avec Supabase.

Mission 3.1 : JSONB pour Métadonnées IoT

Contexte : PostgreSQL a un type JSONB (binaire) ultra-performant. Parfait pour des métadonnées capteurs qui évoluent.

MISSION : Comprendre JSON vs JSONB

1. Quelle est la différence entre JSON et JSONB dans PostgreSQL ?

Piste de recherche : Cherchez 'PostgreSQL JSON vs JSONB'. JSONB est binaire, compressé, indexable.

2. Pourquoi JSONB est-il plus rapide ?

Comprendre le stockage binaire vs texte.

MISSION : Ajouter metadata JSONB à bins

1. Comment ajouter une colonne JSONB ?

Piste de recherche : ALTER TABLE ... ADD COLUMN ... JSONB

2. Comment insérer des données JSONB ?

Piste de recherche : Cherchez 'PostgreSQL JSONB insert'. Vous pouvez utiliser json_build_object() ou du JSON littéral.

3. Comment requêter un champ dans JSONB ?

Piste de recherche : Cherchez 'PostgreSQL JSONB operators'. Regardez -> vs ->> et @>.

OPÉRATEURS JSONB IMPORTANTS

-> retourne du JSON

->> retourne du texte
@> vérifie si contient
? vérifie si clé existe
?| vérifie si au moins une clé existe

MISSION : Indexer JSONB pour la performance

1. Quel type d'index pour JSONB ?

Piste de recherche : Cherchez 'PostgreSQL GIN index JSONB'. GIN est optimal pour JSONB.

2. Comment créer un index GIN sur une colonne JSONB ?

3. Comment indexer un champ SPÉCIFIQUE dans JSONB ?

Piste de recherche : Cherchez 'PostgreSQL index JSONB field'. Vous pouvez créer un index sur (metadata->>'field').

Validation : Comparez EXPLAIN ANALYZE avant/après index GIN sur une requête filtrant metadata. Le Seq Scan doit devenir Index Scan.

À rendre : Fichier jsonb_integration.md : choix JSON/JSONB justifié, opérations réalisées, index créés, gains mesurés

Mission 3.2 : Historisation avec Triggers PostgreSQL

Contexte : Quand un agent change de zone, vous devez garder l'historique. PostgreSQL peut automatiser cela avec des triggers.

MISSION : Concevoir la table users_history

Structure SCD Type 2 :

- user_id (stable, identifiant métier)
- Colonnes métier (email, name, role, zone)
- valid_from, valid_to (timestamps)
- is_current (boolean)

Pourquoi cette structure ? Comment répondre à :

- Quel était le rôle de user 123 le 15 mars 2025 ?
- Quand user 456 a-t-il changé de zone ?

MISSION : Automatiser avec un trigger

1. Qu'est-ce qu'un trigger PostgreSQL ?

Piste de recherche : Cherchez 'PostgreSQL triggers'. Un trigger exécute une fonction avant/après INSERT/UPDATE/DELETE.

2. Comment créer une fonction trigger en PL/pgSQL ?

Piste de recherche : Cherchez 'PostgreSQL trigger function'. Syntaxe : CREATE FUNCTION ... RETURNS trigger AS $$

3. Logique du trigger pour SCD Type 2 :

Quand UPDATE sur users :

- Fermer l'ancienne version (is_current=false, valid_to=NOW)
- Insérer une nouvelle version (is_current=true, valid_from=NOW)

Piste de recherche : Cherchez 'PostgreSQL SCD Type 2 trigger example'. Regardez des exemples complets.

Validation : Testez : UPDATE un utilisateur, vérifiez que 2 lignes existent (ancienne avec is_current=false, nouvelle avec is_current=true).

À rendre : Fichier historisation.md : structure users_history, code du trigger, tests réalisés

Mission 3.3 : Row Level Security (RLS) avec Supabase

Contexte : Supabase implémente RLS nativement. Vous pouvez sécuriser au niveau des lignes, pas juste des tables.

MISSION : Comprendre Row Level Security

1. Qu'est-ce que RLS dans PostgreSQL ?

Piste de recherche : Cherchez 'PostgreSQL Row Level Security'. C'est un filtrage automatique au niveau des lignes selon l'utilisateur.

2. Comment Supabase utilise RLS ?

Piste de recherche : Cherchez 'Supabase Row Level Security'. Supabase se base sur auth.uid() pour identifier l'utilisateur.

MISSION : Implémenter RLS sur bins

Cas d'usage : Les collecteurs ne doivent voir que les poubelles de leur zone.

1. Comment activer RLS sur une table ?

Piste de recherche : Cherchez 'PostgreSQL ALTER TABLE ENABLE ROW LEVEL SECURITY'.

2. Comment créer une policy pour SELECT ?

Piste de recherche : Cherchez 'PostgreSQL CREATE POLICY'. Syntaxe : CREATE POLICY nom ON table FOR SELECT USING (condition).

3. Comment utiliser auth.uid() dans Supabase ?

Piste de recherche : Cherchez 'Supabase auth.uid RLS examples'. auth.uid() retourne l'ID de l'utilisateur connecté.

Exemple de policy à implémenter :

- Les collecteurs voient uniquement bins de leur assigned_zone
- Les admins voient tout
- Les utilisateurs non authentifiés ne voient rien

Validation : Testez avec différents utilisateurs via Supabase. Vérifiez que chacun voit uniquement ses données autorisées.

À rendre : Fichier rls_policies.md : policies créées, logique de sécurité, tests avec différents rôles

PARTIE 4 : DEVOPS & AUTOMATISATION

Objectif : Versionner le schéma, automatiser la maintenance, utiliser les fonctionnalités Supabase.

Mission 4.1 : Versioning avec Migrations Supabase

Contexte : Supabase intègre un système de migrations. Chaque changement de schéma doit être versionné.

MISSION : Comprendre les migrations Supabase

1. Où sont stockées les migrations dans Supabase ?

Piste de recherche : Cherchez 'Supabase migrations'. Dossier supabase/migrations/ dans votre projet.

2. Comment créer une nouvelle migration ?

Piste de recherche : Cherchez 'Supabase CLI migration create'. Commande : supabase migration new nom_migration

3. Comment appliquer les migrations ?

Piste de recherche : Cherchez 'Supabase migration up'. En local vs en prod.

MISSION : Créer vos migrations

Créez des migrations pour :

- Migration 001 : Schéma initial (bins, measurements, etc.)
- Migration 002 : Ajout colonnes JSONB
- Migration 003 : Création des index
- Migration 004 : Partitionnement measurements
- Migration 005 : RLS policies

Chaque fichier de migration doit :

- Avoir un timestamp dans le nom

- Contenir du SQL valide
- Être idempotent si possible (IF NOT EXISTS)

À rendre : Dossier supabase/migrations/ avec tous vos fichiers SQL + README expliquant comment les appliquer

Mission 4.2 : Fonctions PostgreSQL pour la Logique Métier

Contexte : Au lieu de logique complexe côté application, déléguez à PostgreSQL avec des fonctions.

MISSION : Créer une fonction pour calculer les stats quotidiennes

1. Syntaxe des fonctions PostgreSQL (PL/pgSQL) ?

Piste de recherche : Cherchez 'PostgreSQL CREATE FUNCTION'. Différent des procédures stockées MySQL.

2. Comment retourner plusieurs colonnes ?

Piste de recherche : Cherchez 'PostgreSQL function return table'. RETURNS TABLE(col1 type, col2 type, ...).

3. Comment appeler la fonction ?

SELECT * FROM nom_fonction(params);

Votre fonction doit :

- Calculer les stats d'un jour donné (paramètre DATE)
- Retourner : bin_id, avg_fill_level, max_fill_level, measurements_count
- Utiliser des agrégations sur measurements

MISSION : Automatiser avec pg_cron

1. Qu'est-ce que pg_cron ?

Piste de recherche : Cherchez 'PostgreSQL pg_cron extension'. C'est un scheduler intégré à PostgreSQL.

2. Supabase supporte-t-il pg_cron ?

Piste de recherche : Cherchez 'Supabase pg_cron'. Disponible selon le plan.

3. Comment planifier l'exécution quotidienne ?

Piste de recherche : Cherchez 'pg_cron schedule'. Syntaxe cron classique.

À rendre : Fichier fonctions.sql avec vos fonctions + documentation de l'automatisation

Mission 4.3 : Real-time avec Supabase

Contexte : Supabase permet d'écouter les changements en temps réel via WebSockets. Parfait pour le dashboard ECOTRACK.

MISSION : Configurer Real-time sur measurements

1. Comment activer Real-time sur une table ?

Piste de recherche : Cherchez 'Supabase Realtime'. Via l'interface ou via SQL : ALTER PUBLICATION supabase_realtime ADD TABLE measurements;

2. Côté frontend, comment écouter les changements ?

Piste de recherche : Cherchez 'Supabase realtime subscribe'. Utilisez le SDK JavaScript Supabase.

3. Comment filtrer les events (ex: seulement bin_id=123) ?

Piste de recherche : Cherchez 'Supabase realtime filters'.

Cas d'usage pour ECOTRACK :

- Dashboard qui affiche en temps réel les nouvelles mesures
- Alerte quand une poubelle dépasse 85% de remplissage
- Carte qui se met à jour automatiquement

Validation : Testez : Insérez une mesure via SQL. Le frontend doit recevoir l'event immédiatement.

À rendre : Document realtime_config.md : tables activées, filtres configurés, exemples d'utilisation frontend

LIVRABLES & ÉVALUATION

Ce Que Vous Devez Rendre

Un dossier ZIP ECOTRACK_NOM_PRENOM.zip contenant :

1. Documentation (dossier docs/)

- audit_postgresql.md
- requetes_lentes.md
- projection.md
- optimisation_index.md
- partitionnement.md
- jsonb_integration.md
- historisation.md
- rls_policies.md
- config_postgresql.md
- realtime_config.md

2. Code SQL (dossier supabase/)

- supabase/migrations/ (tous vos fichiers SQL numérotés)
- supabase/functions.sql (fonctions PL/pgSQL)
- supabase/policies.sql (toutes vos RLS policies)

3. Rapport de Performance (rapport_performance.pdf)

Document de synthèse avec :

- Tableaux comparatifs AVANT/APRÈS optimisations
- Captures EXPLAIN ANALYZE avant/après
- Graphiques de temps d'exécution
- Analyse : qu'est-ce qui a eu le plus d'impact ?
- Difficultés et solutions

4. Présentation (présentation.pdf - 5 slides max)

Slides pour présentation orale :

- Slide 1 : Problèmes identifiés
- Slide 2 : Solutions techniques
- Slide 3 : Résultats chiffrés
- Slide 4 : Architecture finale
- Slide 5 : Apprentissages

BONUS

Implémentations avancées au choix :
- Extension PostGIS pour géolocalisation
- Monitoring avec pg_stat_statements
- API Supabase Edge Functions
- Contribution innovante

CONSEILS POUR RÉUSSIR

Spécificités PostgreSQL vs MySQL

Concept	MySQL	PostgreSQL
Auto-increment	AUTO_INCREMENT	SERIAL ou GENERATED ALWAYS
JSON	JSON (texte)	JSONB (binaire, plus rapide)
Index JSON	Colonne virtuelle	Index GIN direct
Procédures	DELIMITER $$	CREATE FUNCTION (PL/pgSQL)
Partition pruning	EXPLAIN PARTITIONS	EXPLAIN (partitions removed)
ENUM	ENUM natif	CREATE TYPE ou CHECK
Sécurité fine	Pas de RLS	Row Level Security

Méthodologie

- Documentez en temps réel dans vos fichiers .md
- Utilisez les migrations Supabase pour tout changement
- EXPLAIN ANALYZE systématiquement avant/après
- Testez d'abord en local (supabase start)
- Posez des questions si bloqué > 30 min
- Entraide encouragée, copie interdite

Ressources Essentielles

- PostgreSQL Docs : https://www.postgresql.org/docs/current/
- Supabase Docs : https://supabase.com/docs
- Supabase YouTube : tutoriels officiels
- Discord Supabase : communauté active
- Supabase CLI : https://supabase.com/docs/guides/cli

Pièges PostgreSQL à Éviter

- Utiliser JSON au lieu de JSONB (moins performant)
- Oublier VACUUM après gros DELETE (tables gonflées)
- Index B-tree sur JSONB (utiliser GIN)
- Trop de connexions simultanées (pool de connexions)
- Modifier shared_buffers sans redémarrage

Commandes PostgreSQL Utiles

Pendant votre travail, ces commandes vous aideront :

- \dt - Lister les tables
- \d nom_table - Décrire une table
- \di - Lister les index
- \df - Lister les fonctions
- \l - Lister les bases de données
- \x - Affichage étendu (pour EXPLAIN)

PostgreSQL + Supabase = Stack puissant pour ECOTRACK

"Apprenez en cherchant, pas en copiant.
La documentation est votre meilleure alliée."

— Karim, INGETIS Paris